Executable Requirements via Knowledge-Guided Multi-Agent Planning

Anonymous Author(s)

Abstract

Natural language requirements are readable for stakeholders but are often difficult to execute directly because they lack explicit action semantics, behavioral mappings, and task-level dependencies. This paper studies the problem of transforming natural language requirements into executable task workflows for intelligent software systems. We outline a knowledge-guided multi-agent planning approach that analyzes requirement intent, constraints, actions, and execution dependencies, then constructs an actionable requirement representation through task decomposition, sequence generation, and human review. The proposed study is evaluated around task generation accuracy, execution completeness, requirement-to-task alignment, and planning consistency.

Keywords

requirements engineering, executable requirements, requirement-to-task transformation, multi-agent planning, task planning, human-in-the-loop

1 Introduction

1.1 Background and Motivation

Natural language remains the dominant form for expressing software requirements, yet stakeholder-readable statements are rarely directly executable by intelligent software agents. Recent work has emphasized requirements as a central interface for AI-driven software engineering [3], but moving from textual intent to executable behavior is still an open problem. This paper focuses on *executable requirements*: requirements that can be mapped to explicit tasks, actions, dependencies, and execution workflows.

1.2 Challenges of Executable Requirements

Natural language requirements often omit operational semantics. They describe expected capabilities but do not specify which actions should be performed, how tasks should be ordered, or how constraints affect execution. LLM-based requirement generation can improve textual coverage, but generated requirements may still lack behavior mappings and task-level structure. This creates a gap between requirement engineering artifacts and agent-executable workflows.

1.3 Multi-Agent Task Planning

We study whether role-based agent collaboration can support task transformation. Instead of treating requirement analysis as a single text generation step, the proposed workflow separates semantic analysis, task planning, and execution-flow checking across specialized agents. Knowledge-guided analysis provides domain concepts, task templates, and planning rules that constrain the transformation process.

1.4 Contributions

This paper makes the following contributions:

- We formulate executable requirements as a research problem in intelligent requirements engineering.
- We propose a knowledge-guided multi-agent workflow for transforming natural language requirements into actionable task representations.
- We define an executable requirement construction process covering task decomposition, action sequencing, requirement-to-task mapping, and execution-flow construction.
- We design an evaluation plan for measuring task generation accuracy, execution completeness, requirement-to-task alignment, and planning consistency.

2 Related Work

2.1 Executable Requirements

Executable requirements aim to reduce the gap between human-readable requirement statements and machine-actionable behavior. Controlled natural language and requirement patterns improve precision while preserving readability [1]. However, such specifications do not always provide task sequences or execution dependencies for intelligent agents.

2.2 Requirement-to-Task Transformation

Requirement-to-task transformation studies how requirement statements can be converted into concrete development, validation, or operational tasks. For this paper, the relevant concern is the production of task-level actions that preserve the original requirement intent.

2.3 Task Planning for Actionable AI Systems

Task planning provides mechanisms for decomposing goals into ordered actions and checking whether a plan can satisfy a target objective. In intelligent software systems, planning is useful when a requirement must be operationalized as a workflow rather than kept as a static textual artifact.

2.4 LLM-based Requirement Engineering

LLM-based methods have been used to extract, refine, and generate software requirements. Requirement-aware generation has shown that explicitly modeling requirements can improve downstream software artifacts [2]. This paper focuses on a narrower question: how generated or stakeholder-written requirements can be transformed into executable task workflows.

2.5 Multi-Agent Collaboration

Multi-agent collaboration can separate analysis responsibilities across specialized roles. For executable requirements, this separation supports independent semantic analysis, planning, and

execution-flow inspection. The proposed work uses multi-agent collaboration as a task-planning mechanism, not as a general-purpose platform architecture.

3 Methodology

3.1 Framework Overview

The proposed method transforms a natural language requirement into an executable requirement representation. The workflow contains four stages: requirement semantic analysis, knowledge-guided task transformation, execution-flow construction, and human review. The output is a structured plan that links each requirement element to tasks, actions, constraints, and dependencies.

3.2 Multi-Agent Task Planning Workflow

3.2.1 Analyst Agent. The analyst agent identifies the functional intent, relevant entities, constraints, and observable behaviors expressed in the requirement. Its output is a semantic summary used by later planning steps.

3.2.2 Planning Agent. The planning agent decomposes the analyzed requirement into candidate tasks and orders them according to dependency and constraint information. It is responsible for constructing an action sequence that can be inspected and executed by an intelligent system.

3.2.3 Execution Agent. The execution agent checks whether the task workflow is complete. It inspects whether the plan contains actionable steps, inputs, outputs, and dependencies.

3.3 Requirement Semantic Analysis

3.3.1 Functional Intent. Functional intent captures the capability or behavior expected by the requirement. It defines the goal that the task workflow must preserve.

3.3.2 Constraints. Constraints capture conditions that restrict task execution, including input conditions, business rules, quality constraints, and stakeholder-specified limitations.

3.3.3 Action Semantics. Action semantics identify the operations implied by the requirement, such as collecting information, validating conditions, updating state, notifying users, or producing outputs.

3.3.4 Execution Dependencies. Execution dependencies describe ordering relations among tasks and actions. They help prevent plans that satisfy isolated actions but fail as complete workflows.

3.4 Knowledge-Guided Task Transformation

3.4.1 Requirement Knowledge. Requirement knowledge provides domain concepts, common requirement types, and terminology used to interpret stakeholder statements.

3.4.2 Task Templates. Task templates define reusable structures for common executable requirement patterns, such as input validation, authorization checks, notification workflows, and state transitions.

3.4.3 Planning Rules. Planning rules constrain task decomposition and ordering. They specify when tasks require preconditions, postconditions, dependency checks, or human confirmation.

3.4.4 Prompt Strategies. Prompt strategies guide the agents to produce structured intermediate artifacts instead of unconstrained natural language summaries. Prompts are designed around intent extraction, action identification, dependency reasoning, and plan consistency checking.

3.5 Executable Requirement Construction

3.5.1 Task Decomposition. The method decomposes each requirement into a set of tasks that are specific enough to be inspected, assigned, or executed by an intelligent agent.

3.5.2 Action Sequence Generation. Tasks are ordered into an action sequence that reflects preconditions, data dependencies, and expected outputs.

3.5.3 Requirement-to-Task Mapping. Each task is linked back to the requirement fragment that justifies it. This mapping supports traceability between stakeholder intent and executable behavior.

3.5.4 Execution Flow Construction. The final representation organizes tasks, actions, constraints, and dependencies into an execution flow. The flow is intended for planning and inspection, not for deployment-level execution.

3.6 Human-in-the-Loop Interaction

Human reviewers inspect ambiguous mappings, missing constraints, and inconsistent task orderings. Their feedback is used to revise the executable requirement representation before evaluation.

4 Experiments

4.1 Research Questions

The evaluation is organized around the following research questions:

RQ1 How accurately can the approach transform natural language requirements into executable tasks?

RQ2 Does knowledge guidance improve alignment and execution completeness?

RQ3 Does multi-agent task planning improve planning consistency compared with simpler transformation baselines?

RQ4 How does human review affect the quality of the final executable requirement representation?

4.2 Experimental Setup

The experiment evaluates the complete requirement-to-task workflow. For each requirement, the approach produces semantic analysis artifacts, task decompositions, action sequences, requirement-to-task mappings, and execution flows. All outputs are compared against manually inspected reference artifacts or expert judgments.

4.3 Datasets

The dataset should contain natural language requirements from course project specifications or representative intelligent software system scenarios. Each requirement should be annotated with expected task units, action semantics, and dependency information. This section will report dataset sources, selection criteria, annotation protocol, and basic statistics once the dataset is finalized.

4.4 Baselines

The planned baselines are:

- Direct prompting that asks an LLM to produce tasks from a requirement in a single step.
- A single-agent planner without role separation.
- A template-only transformation method without knowledge-guided reasoning.
- A variant without human review.

4.5 Evaluation Metrics

4.5.1 Task Generation Accuracy. Task generation accuracy measures whether generated tasks correspond to valid actions implied by the original requirement.

4.5.2 Execution Completeness. Execution completeness measures whether the task workflow covers the necessary steps, inputs, outputs, and dependencies required to operationalize the requirement.

4.5.3 Requirement-to-Task Alignment. Requirement-to-task alignment measures whether each generated task can be traced to a specific requirement fragment without introducing unsupported behavior.

4.5.4 Plan Consistency. Plan consistency measures whether the action sequence respects dependency order, constraints, and expected flow.

4.6 Main Results

This subsection will report quantitative and qualitative results for the research questions. No synthetic performance numbers are included in this structure draft.

4.7 Ablation Study

The ablation study will evaluate the contribution of knowledge guidance, agent role separation, planning rules, task templates, and human review. Each ablation removes one component while keeping the remaining workflow unchanged.

4.8 Case Study

The case study will show one representative requirement and trace its transformation into semantic elements, tasks, action sequences, requirement-to-task mappings, and an execution flow.

5 Threats to Validity

5.1 Internal Validity

Internal validity threats mainly arise from annotation subjectivity and evaluator bias when judging whether a generated task is correct or complete. The study should mitigate these threats through explicit annotation guidelines, independent review, and disagreement resolution.

5.2 External Validity

External validity threats concern whether the findings generalize beyond the selected requirements and software domains. The evaluation should include requirements with different functional intents, constraint types, and workflow complexity, while avoiding claims beyond the studied dataset.

5.3 Construct Validity

Construct validity threats arise if the metrics do not fully capture executability. Task accuracy, execution completeness, alignment, and consistency should therefore be interpreted together rather than as isolated indicators.

6 Conclusion

This paper frames executable requirements as a focused research problem in intelligent requirements engineering. We propose a knowledge-guided multi-agent planning workflow for transforming natural language requirements into actionable task representations and execution plans. The core idea is to connect requirement intent, constraints, action semantics, and dependencies with task decomposition, action sequencing, and requirement-to-task mapping. The planned evaluation examines whether this workflow improves task generation accuracy, execution completeness, alignment, and planning consistency.

References

[1] Ikram Darif, Ghizlane El Boussaidi, Segla Kpodjedo, and Cristiano Politowski. 2026. Controlled Natural Language for Requirements Specification: A Systematic Literature Review. *ACM Comput. Surv.* 58, 7 (Jan. 2026), 175:1–175:36. doi:10.1145/3778169

[2] Hojae Han, Jaejin Kim, Jaeseok Yoo, Youngwon Lee, and Seung-won Hwang. 2024. ArchCode: Incorporating Software Requirements in Code Generation with Large Language Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 13520–13552.

[3] Diana Robinson, Christian Cabrera, Andrew D. Gordon, Neil D. Lawrence, and Lars Mennen. 2025. Requirements Are All You Need: The Final Frontier for End-User Software Engineering. *ACM Trans. Softw. Eng. Methodol.* 34, 5 (May 2025), 141:1–141:22. doi:10.1145/3708524

A Appendix

The appendix can include the final annotation guideline, prompt templates, task schema, and additional case-study examples used in the evaluation. It should not introduce new research claims beyond the main paper.